

THE 7-STEP SYSTEM DESIGN FRAMEWORK

A deep reference — what each step is, why it exists, and exactly what to say

Worked end-to-end with one running example: "Design Twitter/X"

How to read this document

Each step has five parts: (1) **What it is** in one line, (2) **Why it exists** — the failure it prevents, (3) **What you actually do**, (4) **The running Twitter example**, and (5) a **script box** showing roughly what it sounds like spoken aloud. Read it once start-to-finish, then keep it open beside your first 3–4 practice designs and consciously walk the steps. After ~4 reps you won't need it.

The time budget for a 45-minute interview is shown per step. Total ≈ 45 min. The single biggest pacing mistake is spending 15 minutes on requirements and never reaching the deep dive — the timings exist to prevent exactly that.

The Framework at a Glance

Memorize this column of seven words first — **Requirements, Estimates, API, Data, High-level, Deep-dive, Tradeoffs**. Everything else is detail you hang on these seven pegs.

#	Step	One-line purpose	Time
1	Requirements	Decide what you're building and at what scale, before designing anything	~5 min
2	Estimates	Put rough numbers on scale so later choices are justified, not arbitrary	~5 min
3	API design	Define the contract; this anchors the data model and components	~5 min
4	Data model	Entities, schema, and the storage choice — with a reason tied to access patterns	~7 min
5	High-level design	The boxes-and-arrows diagram; walk the request path end to end	~8 min
6	Deep dive	Go deep on 1–2 components; show real depth and find the bottleneck	~10 min
7	Tradeoffs / wrap	State tradeoffs, failure modes, monitoring; what you'd do with more time	~5 min

The mental model

Steps 1–2 are **scoping** (you're mostly asking, not designing). Step 3–4 are the **contract and the nouns**. Step 5 is the **skeleton**. Step 6 is where you **earn the hire** — depth. Step 7 is **maturity** — showing you know nothing is free. A weak candidate sprints to step 5 and draws boxes. A strong one earns the right to those boxes through 1–4, then goes deep in 6.

Step 1 — Requirements (~5 min)

What it is

Pinning down **exactly what system you're being asked to build**, and at **what scale**, before you design a single component. It is a conversation, not a monologue — you ask, the interviewer answers, you write it down.

Why it exists (the failure it prevents)

“Design Twitter” is deliberately under-specified. Do they mean the timeline? DMs? Search? Trending? Notifications? If you start designing without scoping, you will either build the wrong thing or try to build everything and run out of time. Interviewers are explicitly evaluating whether you **clarify ambiguity before acting** — it's one of the graded axes, and at big tech it maps directly to a core trait (“deals with ambiguity”). Skipping this step doesn't just lose time; it loses a signal they are actively scoring.

What you actually do

- **Separate functional from non-functional.** Functional = what it does (post a tweet, follow a user, view a feed). Non-functional = the qualities (scale, latency, availability, consistency).
- **Explicitly scope IN and OUT.** Say out loud what you will and won't cover. “I'll focus on posting tweets and the home timeline; I'll treat search and DMs as out of scope unless you'd like me to cover them.” This shows judgment and protects your time.
- **Ask the four questions that change the design:** (a) scale — how many users / how much traffic? (b) read vs write ratio? (c) latency target — is this user-facing real-time or can it be async? (d) consistency — does stale data for a few seconds matter?
- **Write the answers down** (on the whiteboard / shared doc). You will refer back to them in every later step to justify decisions.

Running example — Design Twitter/X

Functional (in scope): a user can post a tweet; a user can follow other users; a user can view a home timeline of tweets from people they follow, newest first.

Out of scope (stated explicitly): search, DMs, trending, ads, notifications — “happy to come back to these if time allows.”

Non-functional: ~200M daily active users; massively **read-heavy** (people read feeds far more than they post — roughly 100:1); home timeline should load in **under ~200 ms**; **eventual consistency is acceptable** (a tweet appearing in followers' feeds a few seconds late is fine; this is a huge simplification and you should say so).

WHAT THIS SOUNDS LIKE OUT LOUD

“Before I design anything, let me clarify scope. When you say ‘design Twitter,’ I'm going to assume the core is: posting tweets, following users, and a home timeline. I'll treat search, DMs, and trending as out of scope unless you want them. A few questions: roughly what scale are we targeting — hundreds of millions of users? And is this read-heavy as I'd expect, where reads massively outnumber writes? And is eventual consistency acceptable for the timeline — is it OK if a tweet shows up in followers' feeds a couple of seconds late?”

“Great — so to summarize what I'm building: [restate the scoped requirements]. I'll keep these on the board and refer back as I make decisions.”

Pitfalls

- **Designing during this step.** Resist. No databases, no boxes yet. Just scope.
- **Asking no questions.** Silence here reads as ‘jumps to solutions without understanding the problem’ — a documented negative signal.

- **Accepting the prompt literally.** 'Design Twitter' in full is a year of work; un-scoped, you'll fail by trying to cover everything.

Step 2 — Estimates (~5 min)

What it is

Quick “back-of-the-envelope” math to attach **rough numbers** to the scale you just clarified: queries per second (QPS), storage growth, bandwidth. Precision is irrelevant — order of magnitude is everything.

Why it exists (the failure it prevents)

Every later decision — do I need a cache, do I need to shard the database, one server or a thousand — is only defensible **relative to a number**. Without estimates, “I’ll add a cache” is a buzzword; with them, “at ~350K timeline reads/sec a single DB can’t serve this, so a cache is mandatory” is engineering. Estimates convert opinions into justified decisions. Interviewers specifically listen for whether your architecture is **driven by the numbers** or decorated with them afterward.

What you actually do

- **Start from the scale figure** you established in step 1 (e.g. 200M DAU).
- **Estimate writes:** DAU × actions per user per day, divided by 86,400 sec/day, to get average QPS. Multiply by ~2–3 for peak.
- **Estimate reads** using the read:write ratio from step 1 (often the dominant number for a feed system).
- **Estimate storage:** writes/day × size per item × 365 → storage/year. Round hard.
- **Say the numbers out loud and write them down.** Then immediately state the implication — that’s the whole point of doing it.

Running example — Design Twitter/X

Writes (tweets): say 200M DAU, each posts ~2 tweets/day → 400M tweets/day ÷ 86,400 ≈ **~4,600 tweets/sec average**, call it ~10K/sec at peak.

Reads (timeline loads): read-heavy at ~100:1 → on the order of **~hundreds of thousands of timeline reads/sec**. This is the number that dominates the design.

Storage: 400M tweets/day × ~300 bytes ≈ 120 GB/day ≈ **~44 TB/year** of tweet text alone (media handled separately, far larger).

The implications you state immediately: (1) read volume is orders of magnitude above write volume → the system must be optimized for reads, caching is not optional; (2) yearly storage is large but not exotic → a sharded store is needed but it’s a known problem; (3) the read:write asymmetry is the single most important fact and it will drive the timeline design in step 6.

WHAT THIS SOUNDS LIKE OUT LOUD

“Let me put rough numbers on this. With ~200 million daily users posting a couple of tweets each, that’s about 400 million tweets a day — roughly 4–5 thousand writes per second average, maybe 10K at peak. But it’s read-heavy, so timeline reads are on the order of hundreds of thousands per second. That asymmetry is the key fact: this is overwhelmingly a read-optimization problem, so caching and how I build the timeline will be the heart of the design. Storage is tens of terabytes a year for text — large, so I’ll need to shard, but that’s a solved problem.”

Pitfalls

- **False precision.** Nobody wants 4,629.6 QPS. ‘A few thousand per second’ is the right resolution. Round aggressively.
- **Numbers with no conclusion.** Computing QPS and then not saying what it implies wastes the step.

Always end with '...therefore'.

- **Spending 12 minutes here.** It's a 5-minute sanity check, not an actuarial exercise.

Step 3 — API Design (~5 min)

What it is

Defining the small set of **operations the system exposes** — the method names, their inputs, and their outputs. Think of it as the contract between the client and your system.

Why it exists (the failure it prevents)

The API is the bridge between “what it does” (step 1) and “how data is shaped” (step 4). Defining it forces precision: to write `getTimeline(userId, pagination)` you must decide it's paginated, which forces you to think about how the feed is stored and retrieved. Candidates who skip this tend to produce vague data models and hand-wave the read path. The API is also where you naturally surface auth, pagination, and idempotency — all things interviewers like to see raised unprompted.

What you actually do

- **List only the core endpoints** that map to your in-scope functional requirements — usually 3 to 5. Don't design a hundred endpoints.
- **Specify inputs and outputs** at a signature level. You do not need full request/response JSON; the shape and key fields are enough.
- **Decide REST vs something else briefly.** REST/HTTP is the safe default; mention gRPC or WebSockets only if the problem needs low latency or push (e.g. chat).
- **Flag the cross-cutting concerns:** pagination for any list endpoint, an idempotency key for writes that must not double-apply, auth via a token. One sentence each.

Running example — Design Twitter/X

```
postTweet(userId, content) → { tweetId, timestamp }
```

```
followUser(followerId, followeeId) → { success }
```

```
getHomeTimeline(userId, cursor, limit) → { tweets[], nextCursor }
```

Notes you say aloud: the timeline endpoint is **cursor-paginated** (not offset — offset breaks on a constantly-changing feed); `postTweet` should accept an **idempotency key** so a retried request doesn't create duplicate tweets; all endpoints are authenticated via a token resolved to `userId` at the gateway.

WHAT THIS SOUNDS LIKE OUT LOUD

“The core API is small. `postTweet` takes a user and content and returns a tweet ID and timestamp. `followUser` is straightforward. The important one is `getHomeTimeline` — it takes a user and a pagination cursor and returns a page of tweets plus the next cursor. I'm using cursor-based pagination deliberately, because offset pagination breaks when the underlying feed is changing every second. I'd also have `postTweet` accept an idempotency key so a client retry doesn't double-post.”

Pitfalls

- **Designing 20 endpoints.** Stay to the in-scope core; breadth here is wasted time.
- **Offset pagination on a live feed.** Classic mistake; cursor-based is the expected answer.
- **Full JSON schemas.** Signatures and key fields are enough; verbosity costs you the clock.

Step 4 — Data Model (~7 min)

What it is

The **entities** the system stores, their key **fields and relationships**, and the **storage technology choice** (SQL vs a specific NoSQL family) — with the choice **justified by the access pattern**, never by fashion.

Why it exists (the failure it prevents)

This is where many otherwise-good candidates lose the round, by saying “I’ll use NoSQL because it scales.” That is a non-answer. The data model is graded on whether your storage choice **follows from how the data is read and written**. The interviewer wants to see you reason: “the access pattern is X, therefore this store, with this as the partition key, indexed this way.” It also sets up step 6 — the bottleneck is almost always here.

What you actually do

- **List the core entities** and their important fields: e.g. User, Tweet, Follow (the relationship).
- **State the access patterns first**, then choose storage to fit them. “I need to look up tweets by author, and fetch a user’s followers fast” drives the choice — not the other way around.
- **Pick storage with a stated reason**. SQL when relationships/transactions matter and scale is moderate; a key-value or wide-column store when you need massive horizontal scale on a known access pattern; name the actual tradeoff.
- **Specify the partition/shard key and key indexes**. This is the detail that separates senior-sounding answers — “shard tweets by authorId so a user’s tweets are co-located.”

Running example — Design Twitter/X

Entities: User (id, handle, name); Tweet (id, authorId, content, createdAt); Follow (followerId, followeeId, createdAt).

Access patterns: (1) write a tweet; (2) get a user’s recent tweets by authorId; (3) get a user’s followers / followees; (4) assemble a home timeline (the hard one — deferred to step 6).

Storage choice with reasoning: the Tweet store is write-heavy, append-mostly, and accessed by a known key (authorId + time) at huge scale → a **wide-column / key-value store** sharded by authorId fits the access pattern and scales horizontally. The Follow graph is relationship data with simpler patterns; it can live in a sharded relational store or a dedicated graph/KV structure. “I’m choosing the wide-column store specifically because the dominant access is ‘give me tweets for these author IDs, newest first’ — a partition-key + sort-key query it’s perfect for, **not** because ‘NoSQL scales.’”

WHAT THIS SOUNDS LIKE OUT LOUD

“Three entities: User, Tweet, and the Follow relationship. Before I pick a database I’ll state the access patterns: I write tweets, I read a given author’s recent tweets, I read a user’s follower list, and I assemble timelines — which I’ll deep-dive separately. Given the dominant pattern is ‘fetch tweets for a set of author IDs, newest first,’ at very high scale, I’ll store tweets in a wide-column store sharded by author ID, so a user’s tweets are co-located and time-sortable. I want to be explicit that I’m choosing this for the access pattern, not because NoSQL is trendy — if the dominant need were multi-entity transactions I’d argue for relational instead.”

Pitfalls

- **‘NoSQL because it scales.’** The single most common fatal hand-wave. Always tie the store to the access pattern.
- **No partition key.** ‘A sharded store’ without saying the shard key is incomplete; the key is the interesting decision.

- **Skipping access patterns.** Choosing a DB before stating how data is read is backwards and reads as inexperience.

Step 5 — High-Level Design (~8 min)

What it is

The **boxes-and-arrows diagram**: client → load balancer → services → data stores, plus the path a request takes through them. The first time you actually draw the system.

Why it exists (the failure it prevents)

This is the shared picture you and the interviewer reason about for the rest of the session. Its purpose is **coverage and a walkthrough**, not depth yet — show that the end-to-end path exists and is coherent. Candidates who jump here first (skipping 1–4) draw a plausible-looking diagram they can't justify; candidates who arrive here having done 1–4 can defend every box. The diagram earns its credibility from the prior steps.

What you actually do

- **Draw the standard spine**: client → load balancer / API gateway → stateless application services → datastores, with a cache alongside the read path.
- **Separate the write path from the read path**. For a feed system these are very different and you'll deep-dive the read path in step 6 — set that up here.
- **Walk one request end to end, out loud**. “A tweet POST hits the gateway, authenticates, goes to the Tweet service, writes to the store, and triggers fan-out...” Narrating the path is the deliverable, not the boxes themselves.
- **Keep components stateless where possible** and say so — it's what makes horizontal scaling trivial and you want that observation on the record.

Running example — Design Twitter/X

Components: Client → API Gateway / LB → (Tweet Service, Timeline Service, Follow Service — all stateless) → Tweet store (sharded), Follow store, plus a Timeline Cache on the read path and a message queue connecting the write path to timeline construction.

Write path: client posts → gateway authenticates → Tweet Service persists the tweet → publishes a “new tweet” event to a queue → (fan-out work happens asynchronously — deep-dived next).

Read path: client requests timeline → gateway → Timeline Service → reads the pre-built timeline from the Timeline Cache (ideally a single fast lookup) → returns a page.

WHAT THIS SOUNDS LIKE OUT LOUD

“Here's the high-level shape. Clients hit a gateway that handles auth and load-balances to stateless services — a Tweet service, a Timeline service, a Follow service. Tweets persist to the sharded store. The key structural decision: the write path and read path are decoupled by a queue. When you post, we persist and publish an event; timeline construction happens asynchronously off that event. When you read your timeline, you ideally hit a pre-computed result in the timeline cache — one fast lookup. Let me trace a tweet through end to end: [walk the write path, then the read path]. Everything except the datastores is stateless, so we scale horizontally by adding instances.”

Pitfalls

- **Drawing this first**. Without steps 1–4 you can't defend the boxes; interviewers probe and it collapses.
- **Boxes with no walkthrough**. The narrated request path is the actual signal, not the picture.
- **Going deep here**. Depth is step 6. Here you want a complete, coherent skeleton — breadth over depth.

Step 6 — Deep Dive (~10 min) — where you earn the hire

What it is

Picking **one or two components** — usually the bottleneck — and going **genuinely deep**: the hard tradeoff, the failure mode, the scaling mechanism. This is the longest step and the one that most determines the outcome at mid-level.

Why it exists (the failure it prevents)

Steps 1–5 prove competence; **step 6 separates a hire from a no-hire**. A surface-level design that never goes deep reads as junior. The interviewer is now probing: “what breaks first, and how do you fix it.” At mid-level, the documented decisive signal is **not the initial design but how you handle this deep dive and its follow-ups**. This is the step to spend your strongest minutes on.

What you actually do

- **Name the bottleneck explicitly** and why it's the bottleneck (point back at your estimates from step 2).
- **Present the real tradeoff** with at least two options, their costs, and a defended choice — not one magic answer.
- **Bring in the mechanisms**: caching, sharding, replication, queues, consistency — and say precisely where and why each applies.
- **Address failure**: what happens when a node/region dies, where data can be lost, how the system degrades.

Running example — Design Twitter/X: the timeline (the classic deep dive)

The bottleneck: from step 2, reads dominate at hundreds of thousands/sec. Building each home timeline on-demand by querying every followee at read time is far too slow. So the core question is **fan-out: when do we do the work?**

Option A — fan-out on read (pull): store tweets only by author; build the timeline at read time by querying all followees and merging. **Pro**: cheap writes, no duplication. **Con**: brutally slow reads for users following many people — and reads are the dominant load. Fails the latency target.

Option B — fan-out on write (push): when a user tweets, immediately push the tweet ID into the precomputed timeline cache of every follower. **Pro**: timeline reads become a single fast lookup — perfect for a read-heavy system. **Con**: a celebrity with 100M followers causes 100M writes per tweet (the “celebrity / hot-key” problem).

The defended answer — hybrid: fan-out on write for the vast majority of users (cheap, gives fast reads); for celebrities above a follower threshold, **don't** fan out — instead the follower's timeline is assembled at read time by merging their precomputed timeline with the small number of celebrities they follow, fetched live. This bounds the worst case on both sides. “This hybrid is the standard real-world answer because it directly trades a manageable amount of read-time merge work to eliminate the unbounded celebrity write amplification.”

Failure & consistency: the fan-out runs off the queue asynchronously, so a fan-out worker dying delays a tweet appearing in some feeds by seconds — acceptable, because in step 1 we established eventual consistency is fine. The queue must be durable so events aren't lost on worker failure; at-least-once delivery plus idempotent timeline insertion handles retries.

WHAT THIS SOUNDS LIKE OUT LOUD

“The bottleneck is the timeline read — from my estimates, reads are hundreds of thousands per second, so I cannot build feeds on demand. The real question is fan-out timing. Option one, fan-out on read: cheap writes

but the read does a huge merge across everyone you follow — and reads are my dominant load, so this fails the latency goal. Option two, fan-out on write: when you tweet I push it into every follower's precomputed timeline, so reads become one fast lookup — ideal for read-heavy — but a celebrity with a hundred million followers means a hundred million writes per tweet. So I'd go hybrid: fan-out on write for normal users, but above a follower threshold I skip fan-out and merge those few celebrity accounts in at read time. That bounds both the write amplification and the read cost. The fan-out is async off a durable queue, which is fine because we agreed eventual consistency is acceptable; I make the timeline insert idempotent so at-least-once retries don't duplicate."

Pitfalls

- **Staying shallow.** 'I'd add a cache' with no mechanism is a no-hire at mid-level. Go to the tradeoff and the failure mode.
- **One magic solution.** Present options and defend a choice; a single unexamined answer looks like memorization.
- **Ignoring failure.** 'What happens when this dies' is the follow-up you must reach before they ask it.

This is your highest-leverage practice target

In your study plan, the five follow-up drills — “what breaks at 10×,” “why this DB,” “node failure,” “consistency risk,” “monitoring” — all live in this step. When you practice, spend the majority of each design's effort here, because this is the step the interview outcome actually turns on.

Step 7 — Tradeoffs & Wrap-Up (~5 min)

What it is

A deliberate closing: state the **tradeoffs you made**, the **failure modes**, how you'd **monitor** the system in production, and what you'd do **with more time**. A short, structured landing — not trailing off.

Why it exists (the failure it prevents)

Junior answers end with "...and that's the design." Senior answers end by acknowledging that **every decision had a cost**. This step signals **engineering maturity**: you know nothing is free, you've thought about operating the thing, not just building it. Mentioning monitoring **unprompted** is a specific, documented strong signal of real production experience — and you have that experience, so make sure it lands.

What you actually do

- **Restate 2-3 key tradeoffs** in one line each: "I chose eventual consistency for timeline freshness to get read latency and availability — the cost is a few seconds of staleness, which we agreed was acceptable."
- **Name the main failure modes** and how the system degrades (not crashes): queue backlog delays feeds; a cache region loss falls back to slower rebuild; etc.
- **Add observability explicitly**: the metrics, logs, and traces you'd watch (timeline read latency p99, fan-out lag, queue depth) and what you'd alert on. Say this even if not asked.
- **End with "with more time I'd..."** naming the things you scoped out in step 1 (search, DMs, media pipeline) — closing the loop shows you never forgot them.

Running example — Design Twitter/X

Tradeoffs: eventual consistency for read speed and availability; the hybrid fan-out trades read-time merge cost to kill celebrity write amplification; denormalized precomputed timelines trade storage and write complexity for fast reads.

Failure modes: fan-out workers down → feeds lag by seconds but recover (degrade, not fail); timeline cache region loss → rebuild from the tweet store, slower but correct.

Monitoring: alert on timeline read p99 latency, fan-out lag (event → visible-in-feed), queue depth, cache hit rate. "I'd treat fan-out lag as a primary SLI since it's the user-visible freshness of the product."

With more time: search (a separate inverted-index service), the media pipeline (object storage + CDN), trending, and DMs — all explicitly scoped out earlier.

WHAT THIS SOUNDS LIKE OUT LOUD

"To wrap up: the main tradeoffs were eventual consistency for read latency and availability, the hybrid fan-out to bound the celebrity case, and denormalized timelines that cost storage and write complexity to make reads fast. On failure: fan-out workers going down degrades freshness by seconds rather than breaking anything, and a cache loss rebuilds from the tweet store. For production I'd monitor timeline read p99, fan-out lag — which I'd treat as the key freshness SLI — queue depth, and cache hit rate, with alerts on each. With more time I'd design the pieces I scoped out: search, the media/CDN path, trending, and DMs."

Pitfalls

- **Trailing off**. 'Yeah, that's about it' wastes the maturity signal. Land it deliberately.
- **No monitoring**. Omitting observability forfeits a documented strong signal you can easily earn — and that you genuinely have the background for.
- **Forgetting the scoped-out items**. Naming them again proves the step-1 scoping was deliberate, not forgetful.

Putting It Together — The One-Page Mental Loop

When you hear any design prompt, this is the loop that should fire automatically:

- **Scope it** (Req) → **size it** (Est) → **define the contract** (API) → **shape the data** (Model) → **draw the skeleton** (HLD) → **go deep on the bottleneck** (Deep dive) → **land it with tradeoffs** (Wrap).

Steps 1–5 should feel **automatic and fast** — they're structure, and structure is what you memorize so your brain is free for step 6, which is where the actual thinking and the actual evaluation happen. The reason to drill the framework until it's reflexive is precisely so that **under pressure your working memory is spent on the problem, not on remembering what to do next**.

The discipline: for your first 3–4 practice designs, keep this open and announce each step out loud as you enter it ("Now I'll move to the data model..."). It will feel mechanical. That mechanical feeling is the framework being burned in. By the fourth design you'll stop needing the document — and that is exactly when it has done its job.